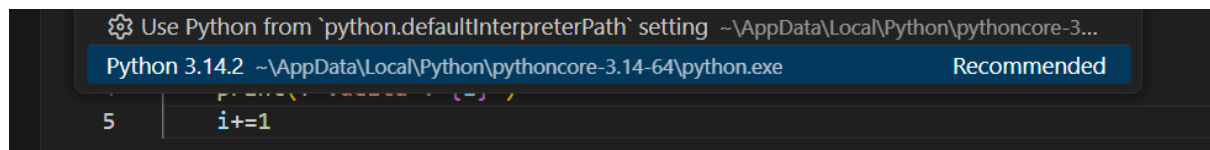
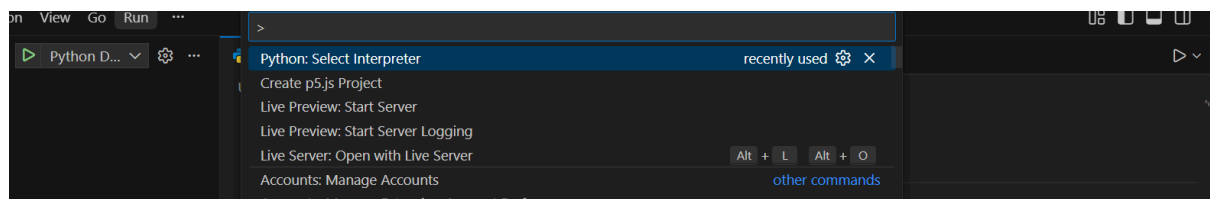


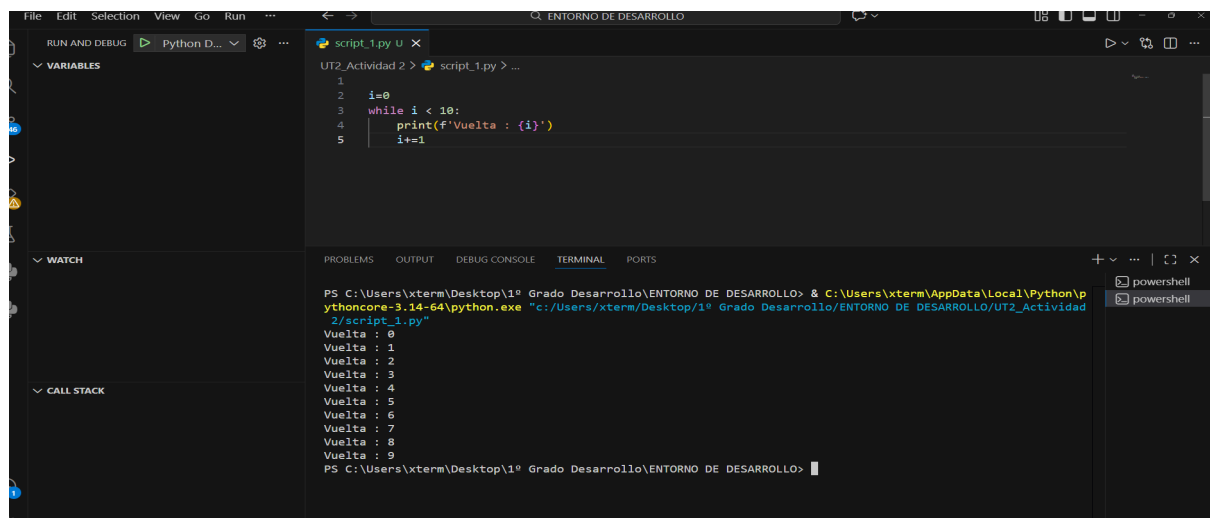
UT2 – Práctica 1: Entornos de Desarrollo (Actividades 1 a 4)

Rama de entrega: ETS_UT2

Parte: 1 (Actividades 1–4)



Abrí el archivo `script_1.py` en Visual Studio Code. El contenido del script es:



¿En qué lenguaje está escrito?

El script está escrito en **Python**.

¿Es un lenguaje compilado o interpretado?

Python es un **lenguaje interpretado**, lo que significa que se ejecuta línea a línea mediante un intérprete.

¿Pude ejecutarlo directamente tras instalar Visual Studio Code?

No. Aunque Visual Studio Code estaba instalado, el entorno requería configurar el intérprete de Python antes de poder ejecutar el script.

¿Qué tuve que instalar para ejecutarlo?

Ya tenía **Python** instalado en el sistema, pero fue necesario **seleccionar el intérprete** dentro de Visual Studio Code para que el entorno lo reconociera.

¿Tuve que instalar algún plugin adicional? ¿Cuál?

Sí. Instalé la extensión oficial de Microsoft llamada **Python**, que permite ejecutar, depurar y analizar código Python dentro de VS Code.

Actividad 3 – Investigación sobre el lenguaje y plugins

Características del lenguaje (Python)

Los scripts proporcionados en la práctica están escritos en **Python**, un lenguaje muy extendido en la actualidad. Sus características principales son:

- **Sintaxis sencilla y muy legible**, ideal para aprender y para desarrollar rápido.
- **Multiplataforma**, funciona en Windows, Linux y macOS.
- **Tipado dinámico**, no es necesario declarar tipos de variables.
- **Multiparadigma**, permite programación orientada a objetos, funcional y estructurada.
- **Gran ecosistema de librerías**, especialmente en ciencia de datos, IA, automatización y backend.
- **Interpretado**, no genera un binario compilado, sino que se ejecuta mediante un intérprete.
- **Comunidad enorme**, con documentación abundante y soporte continuo.

Puntos fuertes del lenguaje

- **Muy fácil de aprender y leer**, lo que acelera el desarrollo.
- **Gran cantidad de librerías** para prácticamente cualquier tarea.
- **Excelente para automatización, scripting y prototipado rápido.**
- **Muy usado en IA, machine learning, análisis de datos y backend web.**
- **Compatible con múltiples IDEs y entornos.**

Puntos débiles

- **Menor rendimiento** que lenguajes compilados como C++ o Java.
- **No es ideal para aplicaciones móviles nativas.**
- **El tipado dinámico puede provocar errores en tiempo de ejecución.**
- **La gestión de dependencias puede complicarse en proyectos grandes.**

Campos de aplicación actuales

Python se utiliza hoy en día en:

- **Inteligencia Artificial y Machine Learning**
- **Ciencia de datos y análisis estadístico**
- **Desarrollo web (Django, Flask)**
- **Automatización de tareas y scripting**
- **Ciberseguridad y pentesting**
- **DevOps y administración de sistemas**
- **Aplicaciones de escritorio (Tkinter, PyQt)**
- **Educación y aprendizaje de programación**

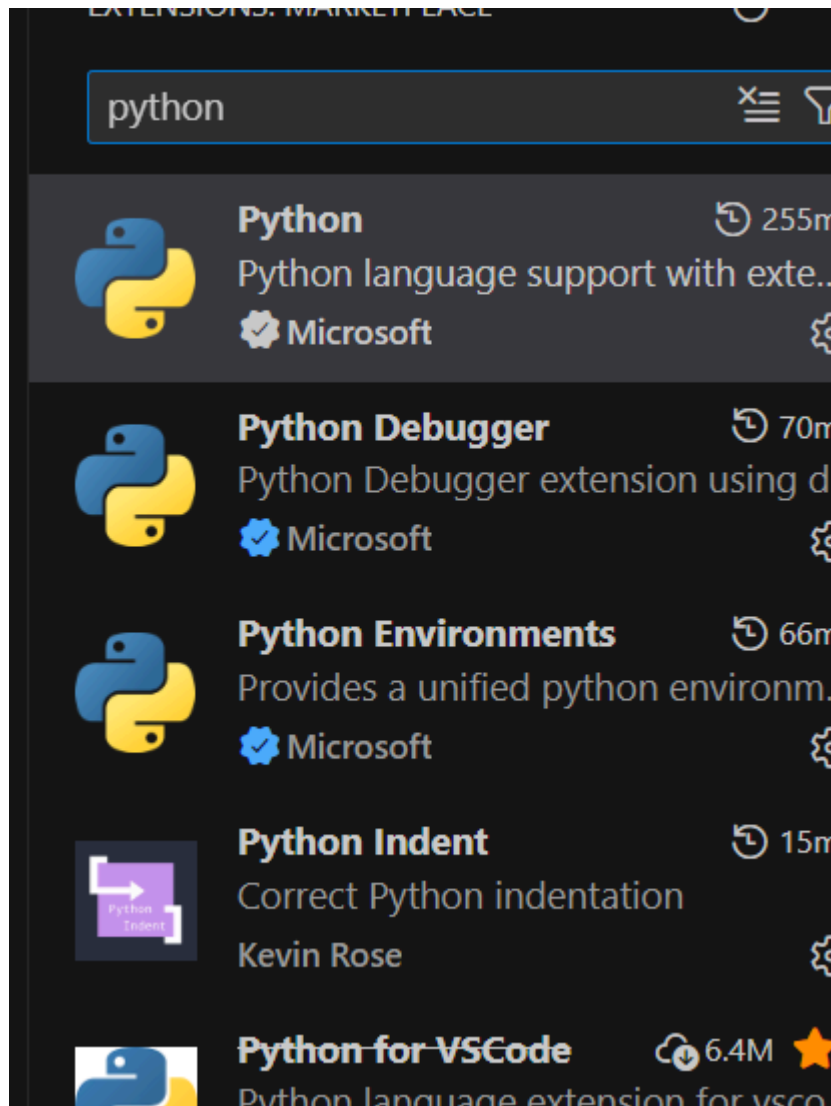
Plugins instalados en Visual Studio Code

Para trabajar de forma más eficiente con Python en VS Code, instalé los siguientes plugins:

1. Python (Microsoft)

Funcionalidad:

- Permite ejecutar y depurar scripts Python.
- Añade autocompletado, análisis de errores y resaltado de sintaxis.
- Integra herramientas como linting, formateo y entornos virtuales.



2. Pylance

Funcionalidad:

- Motor de análisis estático de código.
- Mejora el autocompletado, la detección de errores y el rendimiento del editor.
- Ofrece sugerencias inteligentes y documentación integrada.

Actividad 4.1 – Depuración del script_1.py

El contenido del script es el siguiente:

```
python
```

```
i = 0

while i < 10:

    print(f'Vuelta : {i}')

    i += 1
```

Este programa inicia la variable `i` en 0 y ejecuta un bucle `while` que imprime el número de vuelta en cada iteración hasta que `i` llega a 10.

Punto de ruptura y depuración

Coloqué un **punto de ruptura** en la línea del `while` para detener la ejecución justo antes de que empiece el bucle.

Al iniciar el modo **Debug**, pude observar:

- El valor inicial de `i` en el panel de variables
- Cómo cambiaba `i` en cada iteración
- Cómo se imprimía cada vuelta en la consola

Diferencias entre los modos de ejecución

1. Continue

- Ejecuta el programa de forma normal hasta el siguiente punto de ruptura.
- En este script, como solo hay un breakpoint, avanza directamente hasta terminar el bucle.

2. Step Over

- Ejecuta línea a línea sin entrar en funciones.
- Como este script no tiene funciones, avanza por cada iteración mostrando el valor de `i`.

3. Step Into

- Entra dentro de funciones si las hubiera.
- En este caso, se comporta igual que Step Over porque no hay funciones definidas.

4. Step Out

- Saldría de una función si estuviéramos dentro de una.
- Aquí simplemente continúa la ejecución hasta finalizar el script.

Actividad 4.2 – Depuración del script_2.py

El contenido del script es:

```
python
```

```
def sacarPorPantalla(numero):
```

```
    print(f'Vuelta : {i}')
```

```
i = 0
```

```
while i < 10:
```

```
    sacarPorPantalla(i)
```

```
    i += 1
```

¿Qué hace este script?

El programa intenta imprimir el número de vuelta llamando a una función `sacarPorPantalla(numero)` que recibe un parámetro.

Sin embargo, dentro de la función se imprime:

```
python
```

```
print(f'Vuelta : {i}')
```

Aquí aparece el problema.

Error detectado

Dentro de la función se usa la variable `i`, pero esa variable **no existe dentro del ámbito de la función**.

En Python, las funciones solo pueden acceder a:

- variables locales
- variables globales (si existen)
- parámetros recibidos

En este caso, la función recibe **numero**, pero no lo usa.

Resultado:

Cuando ejecutas el script, aparece un error:

Código

NameError: name 'i' is not defined

Depuración paso a paso

Al colocar un **punto de ruptura** dentro de la función y ejecutar en modo Debug, se observa:

- La función recibe correctamente el valor de **i** como parámetro (**numero**).
- Pero al llegar a `print(f'Vuelta : {i}')`, la variable **i** **no existe en el ámbito local**.
- El panel de variables muestra únicamente:
 - **numero** con su valor
 - variables globales, pero **i no está accesible dentro de la función** en ese momento



Solución correcta

La función debe usar el parámetro que recibe:

python

```
def sacarPorPantalla(numero):
```

```
    print(f'Vuelta : {numero}')
```

Con esta corrección, el script funciona como debería

The screenshot shows a Python IDE with two tabs: 'script_1.py U' and 'script_3.py U X'. The active tab 'script_3.py U X' displays the following code:

```
1 colores = ["rojo", "amarillo", "naranja", "azul", "morado"]
2
3 for i in range(len(colores) + 1):
4     print(colores[i])
```

Below the code editor, the 'TERMINAL' panel shows the execution output and error:

```
Vuelta : 6
Vuelta : 7
Vuelta : 8
Vuelta : 9
PS C:\Users\xterm\Desktop\1º Grado Desarrollo\ENTORNO DE DESARROLLO> & C:\Users\xterm\AppData\Local\Python\p
ythoncore-3.14-64\python.exe "c:/Users/xterm/Desktop/1º Grado Desarrollo/ENTORNO DE DESARROLLO/UT2_Actividad
2/script_3.py"
rojo
amarillo
naranja
azul
morado
Traceback (most recent call last):
  File "c:\Users\xterm\Desktop\1º Grado Desarrollo\ENTORNO DE DESARROLLO\UT2_Actividad 2\script_3.py", line
  4, in <module>
    print(colores[i])
          ~~~~~^^^
IndexError: list index out of range
PS C:\Users\xterm\Desktop\1º Grado Desarrollo\ENTORNO DE DESARROLLO>
```

Actividad 4.3 – Ejecución del script_3.py

El script contiene una lista de colores y un bucle que recorre sus elementos. Sin embargo, el bucle está definido como `range(len(colores) + 1)`, lo que provoca que se intente acceder al índice 5, que no existe en la lista (sus índices válidos son del 0 al 4).

Al ejecutar el programa en modo normal aparece el error:

Código

IndexError: list index out of range

Mediante la depuración pude comprobar que:

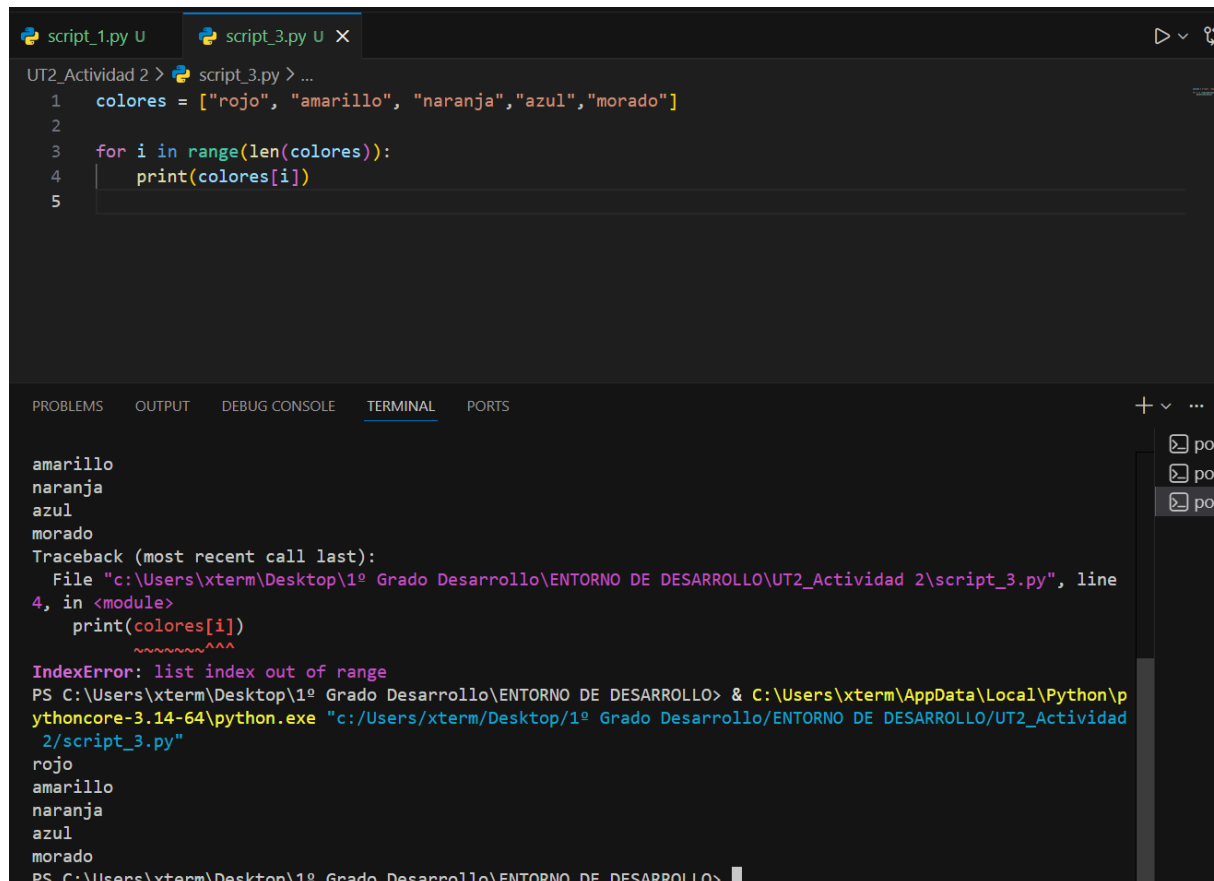
- El bucle funciona correctamente hasta `i = 4`.
- Cuando `i = 5`, el panel de variables muestra que el índice está fuera del rango.
- La ejecución se detiene al intentar acceder a `colores[5]`.

Solución aplicada

Eliminé el **+1** del rango:

```
python
for i in range(len(colores)):
    print(colores[i])
```

Tras corregirlo, el script funciona correctamente.



The screenshot shows a code editor with a file named `script_3.py` open. The code defines a list `colores` with five elements: `["rojo", "amarillo", "naranja", "azul", "morado"]`. A `for` loop iterates over the indices of this list using `range(len(colores))`, and prints each color. The terminal output shows the colors being printed: `amarillo`, `naranja`, `azul`, and `morado`. A `Traceback` message is visible, indicating an `IndexError: list index out of range` at line 4, in `<module>`. The command prompt shows the script was executed using `python.exe` from the directory `c:/Users/xterm/Desktop/1º Grado Desarrollo/ENTORNO DE DESARROLLO/UT2_Actividad 2/`.

```
script_1.py U script_3.py U X
UT2_Actividad 2 > script_3.py > ...
1 colores = ["rojo", "amarillo", "naranja", "azul", "morado"]
2
3 for i in range(len(colores)):
4     print(colores[i])
5

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
amarillo
naranja
azul
morado
Traceback (most recent call last):
  File "c:\Users\xterm\Desktop\1º Grado Desarrollo\ENTORNO DE DESARROLLO\UT2_Actividad 2\script_3.py", line
4, in <module>
    print(colores[i])
      ~~~~~^
IndexError: list index out of range
PS C:\Users\xterm\Desktop\1º Grado Desarrollo\ENTORNO DE DESARROLLO> & C:\Users\xterm\AppData\Local\Python\p
ythoncore-3.14-64\python.exe "c:/Users/xterm/Desktop/1º Grado Desarrollo/ENTORNO DE DESARROLLO/UT2_Actividad
2/script_3.py"
rojo
amarillo
naranja
azul
morado
PS C:\Users\xterm\Desktop\1º Grado Desarrollo\ENTORNO DE DESARROLLO>
```

Actividad 4.4 – Ejecución del script_4.py

El script contiene dos errores distintos. El primero aparece al intentar sumar un string con un número, y el segundo aparece al intentar acceder a un índice fuera del rango de la lista.

Al ejecutar el programa en modo normal aparece:

Código
TypeError: can only concatenate str (not "int") to str

Este error ocurre porque la línea:

```
python
resultado = colores[i] + 1
```

intenta sumar una cadena con un entero.

Después, si se comenta esa línea, aparece un segundo error:

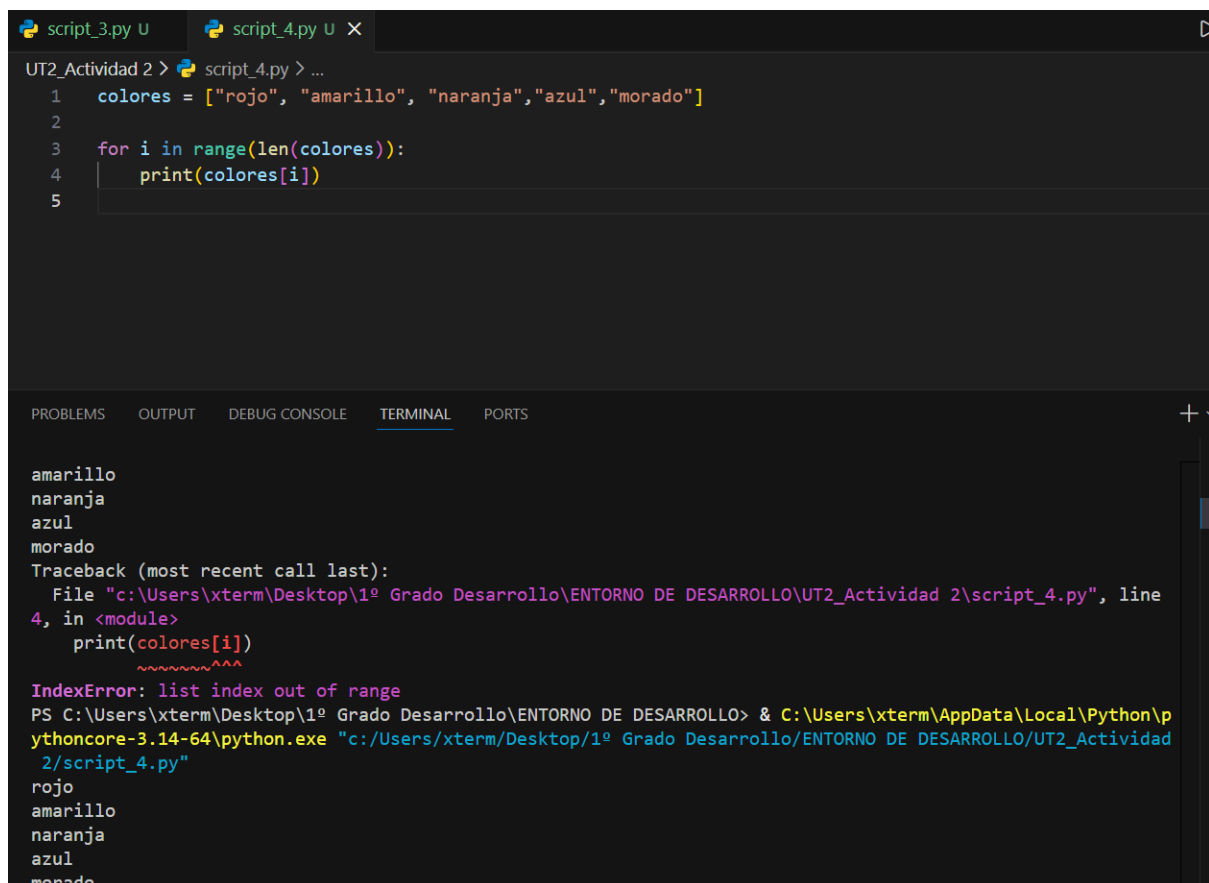
Código

IndexError: list index out of range

Esto ocurre porque el bucle usa `range(len(colores) + 1)`, lo que provoca que se intente acceder al índice 5, que no existe en la lista.

Solución aplicada

- Eliminé el `+1` del rango para evitar el acceso fuera de rango.
- Corregí la operación con strings para evitar el TypeError.



```
script_3.py U script_4.py U X
UT2_Actividad 2 > script_4.py > ...
1 colores = ["rojo", "amarillo", "naranja", "azul", "morado"]
2
3 for i in range(len(colores)):
4     print(colores[i])
5

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

amarillo
naranja
azul
morado
Traceback (most recent call last):
  File "c:\Users\xterm\Desktop\1º Grado Desarrollo\ENTORNO DE DESARROLLO\UT2_Actividad 2\script_4.py", line
4, in <module>
    print(colores[i])
          ~~~~~^^^
IndexError: list index out of range
PS C:\Users\xterm\Desktop\1º Grado Desarrollo\ENTORNO DE DESARROLLO> & C:\Users\xterm\AppData\Local\Python\p
ythoncore-3.14-64\python.exe "c:/Users/xterm/Desktop/1º Grado Desarrollo/ENTORNO DE DESARROLLO/UT2_Actividad
2/script_4.py"
rojo
amarillo
naranja
azul
morado
```

Actividad 4.5 – Ejecución del script_5.py

Este script define una función llamada `operaciones(a, b)` que realiza varios cálculos intermedios y devuelve un resultado final. El programa funciona correctamente y no contiene errores.

Mediante la depuración pude observar cómo iban cambiando los valores de las variables dentro de la función:

- `resultado_intermedio` toma el valor de `a * 2`
- `resultado_intermedio_2` toma el valor de `resultado_intermedio * 10`
- `resultado_final` es la suma de los valores anteriores más `b`

Probé los distintos modos de depuración:

- **Step Into** me permitió entrar dentro de la función y ver cada cálculo.
- **Step Over** ejecutó cada línea sin entrar en otras funciones.
- **Step Out** salió de la función mostrando el valor devuelto.
- **Continue** ejecutó el programa hasta el final.

El resultado final mostrado por el programa fue:

Código

Resultado final: 120

Actividad 4.6 – Ejecución del script_6.py

Este script solicita dos valores por teclado, los convierte a enteros y los pasa a la función `operaciones(a, b)`, que realiza varios cálculos y devuelve un resultado final.

Primero ejecuté el programa introduciendo valores numéricos. El funcionamiento fue correcto y el resultado se mostró sin problemas.

Después probé a introducir letras en lugar de números. En ese caso, Python lanza un error del tipo:

Código

ValueError: invalid literal for int() with base 10

Este error ocurre porque la función `int()` no puede convertir una cadena no numérica en un número entero.

Mediante la depuración pude observar:

- El programa se detiene justo en la línea donde se intenta convertir la entrada a entero.

- El panel de variables muestra que la entrada del usuario es una cadena no válida.
- El error se reproduce siempre que se introducen letras o símbolos.

Para evitar que el programa se cierre de forma abrupta, añadí un bloque `try/except` que captura el error y muestra un mensaje más claro al usuario.

Código corregido

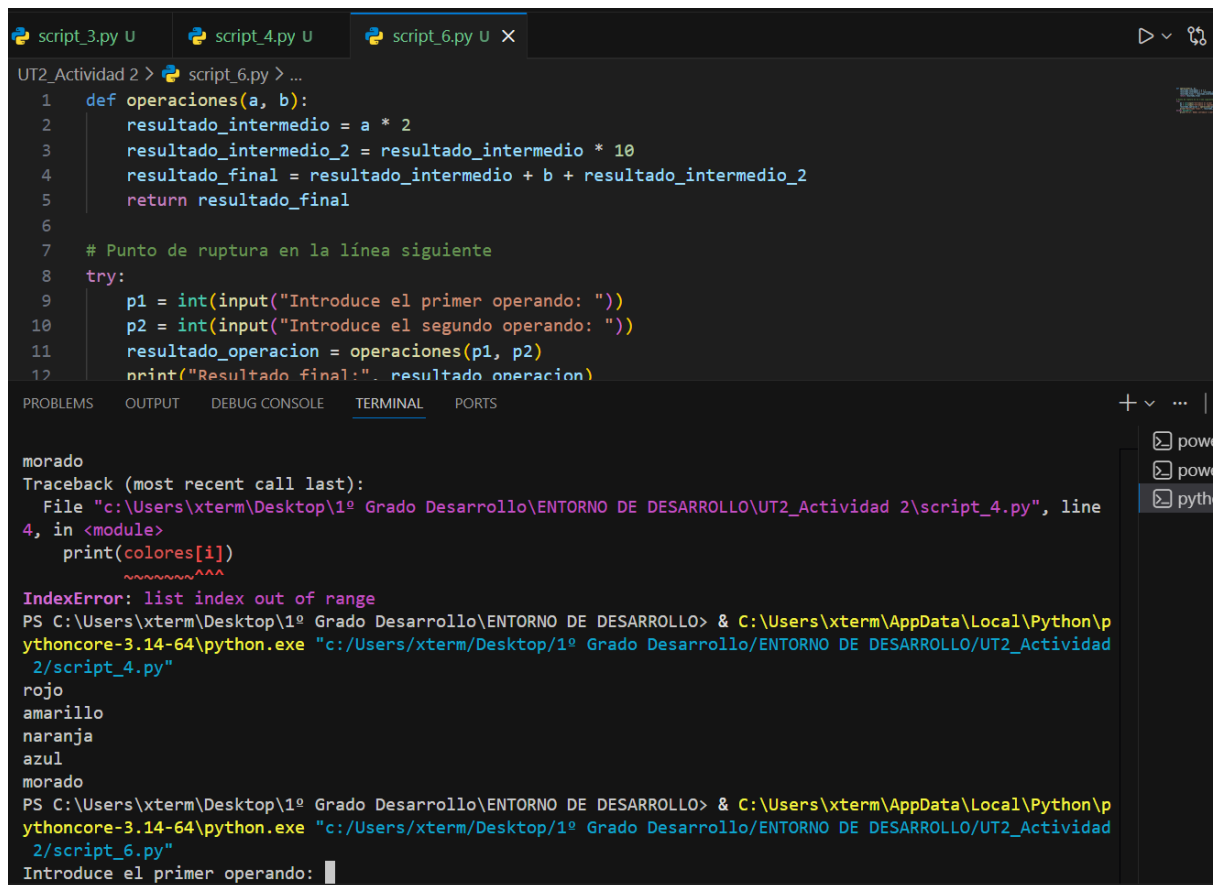
python

try:

```
p1 = int(input("Introduce el primer operando: "))
p2 = int(input("Introduce el segundo operando: "))
resultado_operacion = operaciones(p1, p2)
print("Resultado final:", resultado_operacion)
```

except ValueError:

```
print("Error: Debes introducir números, no letras.")
```



The screenshot shows a Python IDE with three tabs: script_3.py U, script_4.py U, and script_6.py U. The active tab is script_6.py U, which contains the following code:

```
1 def operaciones(a, b):
2     resultado_intermedio = a * 2
3     resultado_intermedio_2 = resultado_intermedio * 10
4     resultado_final = resultado_intermedio + b + resultado_intermedio_2
5     return resultado_final
6
7 # Punto de ruptura en la línea siguiente
8 try:
9     p1 = int(input("Introduce el primer operando: "))
10    p2 = int(input("Introduce el segundo operando: "))
11    resultado_operacion = operaciones(p1, p2)
12    print("Resultado final:", resultado_operacion)
```

The terminal window at the bottom shows the following output:

```
morado
Traceback (most recent call last):
  File "c:\Users\xterm\Desktop\1º Grado Desarrollo\ENTORNO DE DESARROLLO\UT2_Actividad 2\script_4.py", line
  4, in <module>
    print(colores[i])
          ~~~~~^~
IndexError: list index out of range
PS C:\Users\xterm\Desktop\1º Grado Desarrollo\ENTORNO DE DESARROLLO> & C:\Users\xterm\AppData\Local\Python\p
ythoncore-3.14-64\python.exe "c:/Users/xterm/Desktop/1º Grado Desarrollo/ENTORNO DE DESARROLLO/UT2_Actividad
2/script_4.py"
rojo
amarillo
naranja
azul
morado
PS C:\Users\xterm\Desktop\1º Grado Desarrollo\ENTORNO DE DESARROLLO> & C:\Users\xterm\AppData\Local\Python\p
ythoncore-3.14-64\python.exe "c:/Users/xterm/Desktop/1º Grado Desarrollo/ENTORNO DE DESARROLLO/UT2_Actividad
2/script_6.py"
Introduce el primer operando: 
```

Actividad 4.7 – Ejecución del script_7.py

Este script pide al usuario un número, indica si es par o impar mediante la función `checkpar(numero)` y después pregunta si desea continuar o salir del programa.

Primero probé el programa introduciendo números válidos. El funcionamiento fue correcto y el programa detectó correctamente si el número era par o impar.

Después probé a introducir letras en lugar de números. En ese caso, el programa mostró el mensaje:

Código

No has introducido un número

Esto ocurre porque la conversión con `int()` genera un `ValueError`, que está correctamente controlado mediante un bloque `try/except`.

También probé diferentes respuestas a la pregunta “¿Quieres salir? y/n”:

- Si introduzco **y**, el programa termina.
- Si introduzco **n**, el programa continúa pidiendo números.
- Si introduzco cualquier otro valor, el programa entra en un bucle de validación que pide repetidamente “Indica 'y' o 'n'”.

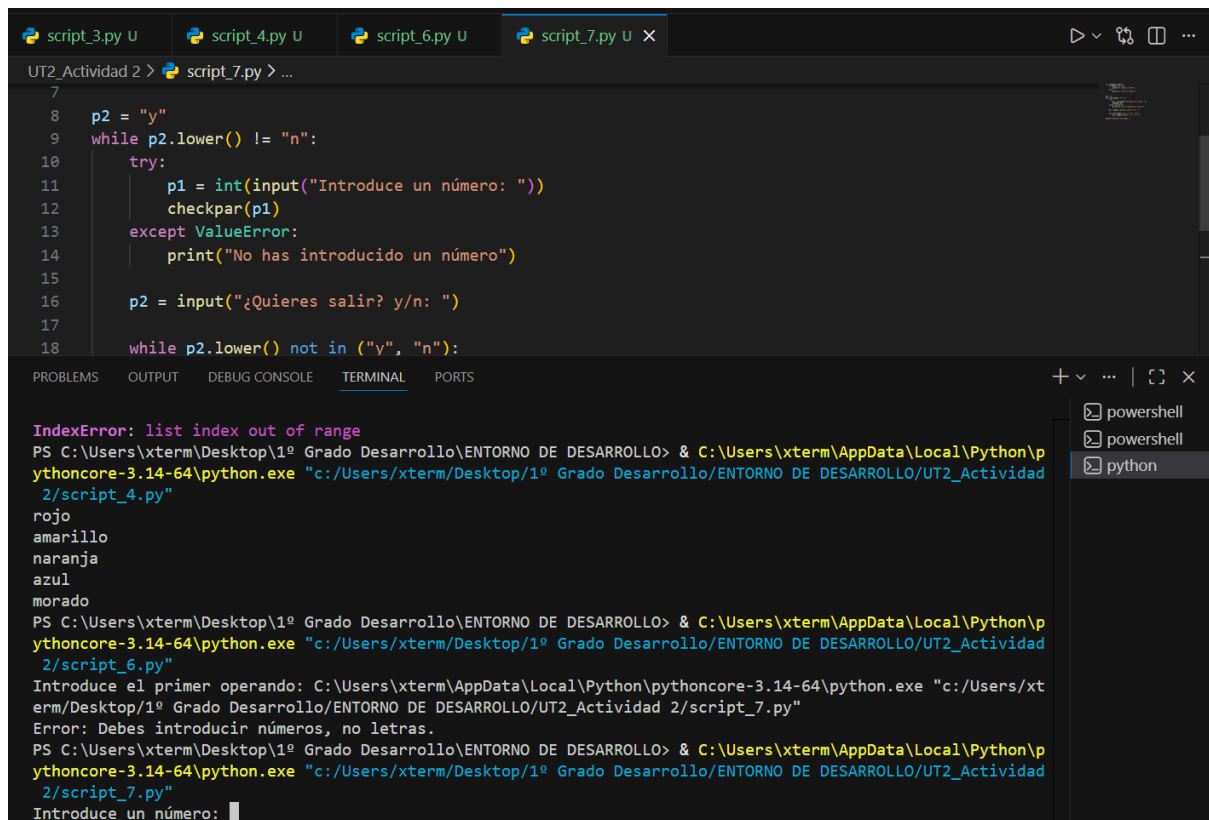
Mediante la depuración pude observar:

- El valor de `p1` cambia correctamente en cada iteración.
- El valor de `p2` controla el flujo del bucle principal.
- El panel de variables muestra claramente cuándo se entra en el bucle de validación.

Para mejorar la estabilidad del programa, simplifiqué el control de entrada de `p2` utilizando un bucle que solo acepta “y” o “n”.

Código corregido

```
python
while p2.lower() not in ("y", "n"):
    p2 = input("Indica 'y' o 'n': ")
```



The screenshot shows a Visual Studio Code editor with a Python script named `script_7.py` open. The script contains a `while` loop that prompts the user to enter a number. The terminal output shows the execution of the script, which prints the colors of the rainbow (rojo, amarillo, naranja, azul, morado) and then prompts the user to enter a number. The terminal also shows the execution of other scripts in the same directory.

```
7
8 p2 = "y"
9 while p2.lower() != "n":
10     try:
11         p1 = int(input("Introduce un número: "))
12         checkpar(p1)
13     except ValueError:
14         print("No has introducido un número")
15
16     p2 = input("¿Quieres salir? y/n: ")
17
18 while p2.lower() not in ("y", "n"):
```

Terminal output:

```
IndexError: list index out of range
PS C:\Users\xterm\Desktop\1º Grado Desarrollo\ENTORNO DE DESARROLLO> & C:\Users\xterm\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/xterm/Desktop/1º Grado Desarrollo/ENTORNO DE DESARROLLO/UT2_Actividad 2/script_4.py"
rojo
amarillo
naranja
azul
morado
PS C:\Users\xterm\Desktop\1º Grado Desarrollo\ENTORNO DE DESARROLLO> & C:\Users\xterm\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/xterm/Desktop/1º Grado Desarrollo/ENTORNO DE DESARROLLO/UT2_Actividad 2/script_6.py"
Introduce el primer operando: C:\Users\xterm\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/xterm/Desktop/1º Grado Desarrollo/ENTORNO DE DESARROLLO/UT2_Actividad 2/script_7.py"
Error: Debes introducir números, no letras.
PS C:\Users\xterm\Desktop\1º Grado Desarrollo\ENTORNO DE DESARROLLO> & C:\Users\xterm\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/xterm/Desktop/1º Grado Desarrollo/ENTORNO DE DESARROLLO/UT2_Actividad 2/script_7.py"
Introduce un número: 
```

Actividad 5 – Ejecución del primer script en Java

El programa define una variable entera `i` inicializada a 0 y ejecuta un bucle `while` que se repite mientras `i` sea menor que 5. En cada iteración se imprime el valor de `i` y luego se incrementa en 1.

El bucle imprime los valores:

Código

```
0
1
2
3
4
```

Cuando `i` llega a 5, la condición `i < 5` deja de cumplirse y el bucle termina. A continuación, el programa ejecuta:

```
java
System.out.println(i);
```

En este punto, el valor de `i` es **5**, ya que ese es el valor que provocó la salida del bucle.

Por tanto, la salida completa del programa es:

Código

```
0
1
2
3
4
5
```

Conclusión

El valor de `i` después del bucle es **5**, porque el incremento final ocurre antes de comprobar la condición de salida.



Actividad 6 – Ejecución del segundo script en Java

El programa intenta imprimir los valores del 0 al 9 utilizando un bucle `for`. Dentro del bucle, la variable `i` está correctamente declarada y funciona sin problemas. Sin embargo, después del bucle se ejecuta:

```
java
System.out.println(i);
```

En el código original, esta instrucción provoca un error de compilación:

Código

`i cannot be resolved to a variable`

Esto ocurre porque la variable `i` está declarada dentro del propio bucle `for`, por lo que su ámbito es únicamente el interior del bucle. Una vez finaliza el `for`, la variable deja de existir.

Solución aplicada

Para que la variable pueda utilizarse después del bucle, la declaré fuera del `for`:

```
java
int i;

for (i = 0; i < 10; i++) {
    System.out.println(i);
}

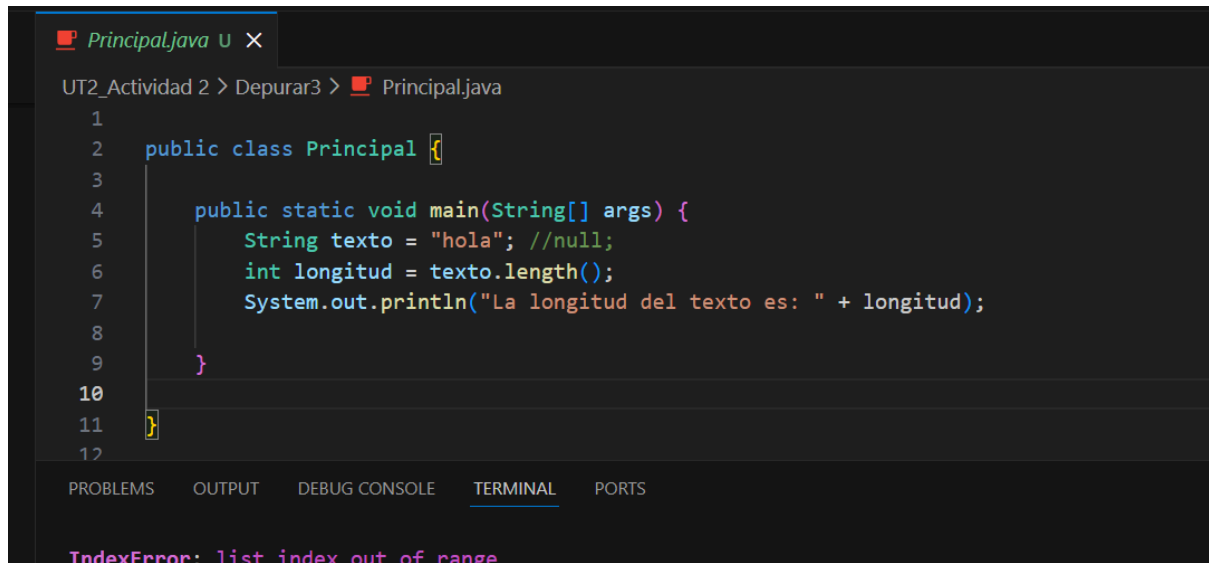
System.out.println(i);
```

Con esta corrección, el programa funciona correctamente y la salida completa es:

Código

```
0
1
2
3
4
5
6
7
8
9
10
```

El valor final impreso es **10**, que es el valor que hace que el bucle termine.



```
Principal.java x
UT2_Actividad 2 > Depurar3 > Principal.java
1
2 public class Principal {
3
4     public static void main(String[] args) {
5         String texto = "hola"; //null;
6         int longitud = texto.length();
7         System.out.println("La longitud del texto es: " + longitud);
8     }
9
10
11 }
12

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
IndexError: list index out of range
```

Actividad 7 – Ejecución del tercer script en Java

El programa declara una variable de tipo `String` llamada `texto` y le asigna el valor `"hola"`. A continuación, obtiene la longitud de la cadena mediante el método `length()` y la imprime por pantalla.

La ejecución con el valor `"hola"` funciona correctamente y muestra:

Código

La longitud del texto es: 4

Después probé a sustituir la línea:

```
java
String texto = "hola";
```

por:

```
java
String texto = null;
```

En este caso, el programa lanza una excepción:

Código

```
java.lang.NullPointerException
```

Esto ocurre porque se intenta ejecutar `texto.length()` cuando `texto` no apunta a ningún objeto. En Java, llamar a un método sobre una referencia `null` siempre provoca un `NullPointerException`.

Mediante la depuración pude observar:

- El valor de `texto` aparece como `null` en el panel de variables.
- La ejecución se detiene exactamente en la línea donde se llama a `.length()`.
- La excepción se lanza antes de que se imprima nada por pantalla.

Conclusión

El método `length()` solo puede usarse sobre cadenas válidas. Si la referencia es `null`, el programa falla con un `NullPointerException`.